

Front-End JavaScript: Make Your Web Pages Come Alive

Every quiz answered and explained, plus reference notes for each lab — for all six modules. Use for grading and reteaching. Confidential instructor material.

6 modules

112 quiz items + solutions

Confidential — do not distribute

CONTENTS

MOD 1 JavaScript Foundations

- L01 Welcome to JavaScript
- L02 Linking Scripts & the Console
- L03 Variables & Data Types
- L04 Operators & Expressions

MOD 2 Control Flow & Functions

- L05 Conditionals
- L06 Loops
- L07 Functions
- L08 Scope & Hoisting

MOD 3 Working with Data

- L09 Arrays
- L10 Objects
- L11 Array Methods
- L12 Strings & Template Literals

MOD 4 The DOM

- L13 Introduction to the DOM
- L14 Selecting & Modifying Elements
- L15 Events & Event Listeners
- L16 Creating & Removing Elements

MOD 5 Building Interactive Pages

L17 Forms & Validation

L18 Timers & Animation

L19 Local Storage

L20 Working with APIs & Fetch

MOD 6 **Capstone & Next Steps**

L21 Building a Complete Project

L22 Next Steps & Best Practices

Module 1 — Answers & Solutions

Correct quiz answers with the "why," plus a reference solution for each hands-on exercise. Use for grading and for reteaching missed items.

Lessons 1–4

20 quiz items

4 exercises

NUMBERING MATCHES THE WORKBOOK

This key covers every question in each lesson's quiz. The participant workbook's self-check uses a 3-question subset per lesson, labelled with the lesson's own question numbers (e.g. Q2 · Q3 · Q4), so a student's "Q4" is always this key's Q4.

L01 • WELCOME TO JAVASCRIPT

Q1

What is JavaScript's primary role in web development?

C) Adding interactivity and behavior

Why: HTML structures and CSS styles; JavaScript responds to clicks, updates content, validates forms. Storing data in databases is a back-end job.

Q2

JavaScript and Java are the same language.

B) False

Why: the name rode Java's 1990s popularity — a marketing decision. They are separate languages with different designs.

Q3

Which of these does a browser need to run JavaScript?

C) Nothing extra — browsers have a built-in JavaScript engine

Why: every modern browser ships an engine (V8, SpiderMonkey, JavaScriptCore). Node.js is for servers; no compiler or plugin is needed.

Q4

In the exercise, what happened when you typed `document.title = "I changed this with JavaScript!"` into the Console?

A) The text in the browser tab changed, just for your copy of the page

Why: `document.title` is the tab text, not the `<h1>`. The change lives only in your browser — a refresh puts it back.

Q5

What is ECMAScript?

D) The official specification (rulebook) that JavaScript follows

Why: engines implement the spec; names like ES6/ES2015 are editions of it. It isn't an engine, a server runtime, or a replacement language.

EXERCISE SOLUTION — SPOT JAVASCRIPT IN THE WILD

Look for: the Console opened (not Elements); `document.title` returned the title as a quoted string; the tab text changed after assignment; the page turned light yellow; a refresh restored both. Their list of three features should be things that change without a full reload (menus, live search, carousels, counters), each with a reason HTML/CSS can't do it. Worksheet and expected results: `labs/module-1/l01-welcome-to-javascript/README.md`; the statements as a script: `labs/module-1/l01-welcome-to-javascript/solution/`.

L02 · LINKING SCRIPTS & THE CONSOLE

Q1

Where is the recommended place to put a `<script>` tag that links to an external file?

B) Just before the closing `</body>` tag

Why: all the HTML has been parsed by then, so every element exists when the script runs. (In the `<head>` with `defer` is also correct — the option says "without defer.")

Q2

What does `console.log()` do?

A) Prints a message to the browser's Developer Tools Console

Why: it's a developer tool — visitors never see it unless they open DevTools. Putting text on the page is Module 4.

Q3

What does the `defer` attribute do on a `<script>` tag?

C) Downloads the script in parallel and runs it after HTML parsing completes

Why: the download doesn't block parsing, and deferred scripts run in order once the document is parsed. It only works on external scripts.

Q4

What happens with this code? `<script src="app.js"> console.log("Hi!"); </script>`

D) app.js runs, but the `console.log()` inside the tags is ignored

Why: when a script has `src`, its contents are ignored — silently, with no error. Use a separate `<script>` for inline code.

Q5

Which of these is a good JavaScript comment?

A) `// Start at 3 lives so new players aren't punished too early`

Why: it explains *why*, which the code can't. B only restates the code; C is HTML comment syntax; D is Python/shell syntax.

EXERCISE SOLUTION — YOUR FIRST JS PROJECT

Look for: a `js-practice` folder with `index.html` and `app.js` side by side; `<script src="app.js">`
`</script>` just before `</body>`; in the Console a log, a yellow warning, a red "not a real one!" error, an info
line, `5 + 3 = 8`, `10 * 4 = 40`, and a table of HTML/CSS/JavaScript. Reference: [labs/module-1/l02-linking-scripts/solution/](#).

L03 · VARIABLES & DATA TYPES

Q1

Which keyword should you use by default when declaring a variable?

C) `const`

Why: it prevents accidental reassignment. Switch to `let` only when the value needs to change; `var` is legacy, and `define` isn't JavaScript.

Q2

What is the result of `typeof null` ?

B) "object"

Why: a bug from 1995 that can't be fixed without breaking old sites. Check for null with `value === null` .

Q3

What does `"5" + 3` evaluate to?

B) "53"

Why: with a string on either side, `+` concatenates: 3 becomes `"3"` and the strings join.

Q4

You write `let favoriteColor;` and then `console.log(favoriteColor);` . What appears in the console?

A) undefined

Why: the variable exists but has no value yet. `null` only appears if you assign it; a `ReferenceError` means the variable was never declared.

Q5

Which of these is a valid JavaScript variable name?

D) `$totalScore`

Why: it starts with `$` and has only letters after. `2ndPlace` starts with a digit, `user-name` has a hyphen, `const` is reserved.

EXERCISE SOLUTION — BUILD A USER PROFILE

Look for: `const` for `firstName` , `lastName` , `email` ; `let` for `age` , `isPremiumMember` , `accountBalance` , `nickname = null` ; seven lines like `age: 30 (number)` (note `nickname: null` (object)); the greeting `Hello, Ray de la Paz! You are 30 years old.` ; updated values ending `Balance: $120.00` via `.toFixed(2)` ; and the `const` reassignment tried, seen to throw `TypeError: Assignment to constant variable` , then commented out. Any names and values are fine if the types and keywords are right. Reference: [labs/module-1/l03-variables-data-types/solution/](https://www.dbooks.org/labs/module-1/l03-variables-data-types/solution/) .

L04 · OPERATORS & EXPRESSIONS

Q1

What does `10 % 3` return?

B) 1

Why: `%` is the remainder: $10 \div 3 = 3$ remainder 1.

Q2

What does `5 === "5"` return?

A) false

Why: strict equality checks value *and* type; a number and a string are different types. (`5 == "5"` would be true.)

Q3

What is the result of `true && false || true`?

B) true

Why: `&&` goes first: `true && false` is `false`; then `false || true` is `true`.

Q4

After this code runs, what is the value of `points`? `let points = 10; points += 5; points *= 2;`

C) 30

Why: `+= 5` makes it 15, then `*= 2` doubles the current value to 30.

Q5

Which of these values is truthy?

D) `"0"`

Why: it's a non-empty string (one character), so it's truthy — even though the number `0` is falsy. `""` and `NaN` are falsy too.

EXERCISE SOLUTION — TIP CALCULATOR

Look for: `const billAmount = 85.50 , tipPercentage = 20 , numberOfPeople = 3 ; tipAmount = billAmount * tipPercentage / 100 ; totalWithTip = billAmount + tipAmount ; perPerson = totalWithTip / numberOfPeople ;` output exactly `Bill: $85.50 , Tip (20%): $17.10 , Total: $102.60 , Per person: $34.20` — each money value through `.toFixed(2)` (without it, per person shows `34.199999999999996`). Bonus: `const isExpensive = totalWithTip > 100; logs Expensive? true`. Reference: labs/module-1/l04-operators-expressions/solution/.

Module 2 — Answers & Solutions

Correct quiz answers with the "why," plus a reference solution for each hands-on exercise. Use for grading and for reteaching missed items.

Lessons 05-08

20 quiz items

4 exercises

NUMBERING MATCHES THE WORKBOOK

This key covers every question in each lesson's quiz. The participant workbook's self-check uses a 3-question subset per lesson, labelled with the lesson's own question numbers (e.g. Q1 · Q4 · Q5), so a student's "Q4" is always this key's Q4.

L05 • CONDITIONALS

Q1

What does this code output?

```
let x = 5;
if (x > 10) {
  console.log("A");
} else if (x > 3) {
  console.log("B");
} else if (x > 1) {
  console.log("C");
} else {
  console.log("D");
}
```

B) B

Why: $5 > 10$ is false, but $5 > 3$ is true, so "B" prints. The chain stops at the first true condition, so $x > 1$ is never checked.

Q2

What happens if you forget `break` in a `switch` case?

B) Execution falls through to the next case

Why: without `break`, the next case's code runs too, whether or not it matches. No error, no jump to `default`.

Q3

What does `let result = 10 > 5 ? "yes" : "no";` assign to `result` ?

A) "yes"

Why: the condition is true, so the ternary returns its first value. It returns one of its two values, never a boolean.

Q4

What does this code output?

```
let input = "5";
switch (input) {
  case 5:
    console.log("number five");
    break;
  default:
    console.log("no match");
}
```

C) no match

Why: `switch` compares with strict equality (`===`); the string `"5"` is not `===` the number `5`, so `default` runs. Convert first with `Number(input)`.

Q5

A shopping cart stores `let quantity = 0;` and zero is a perfectly valid quantity. Why is `if (quantity) { ... }` a risky way to check that a quantity was entered?

D) 0 is falsy, so a valid 0 would be treated as "missing"

Why: `0` is on the falsy list. When zero is legitimate, check explicitly, e.g. `quantity !== undefined`.

EXERCISE SOLUTION — GRADE CALCULATOR

Look for: an invalid-score check first (`score < 0 || score > 100`) wrapping the rest in `else`; an `else if` chain from `>= 90` down storing `letterGrade`; a `switch` on `letterGrade` with a `break` in every case; `const status = score >= 60 ? "Pass" : "Fail"`; output *"Alex scored 78 (C) — Solid effort, keep improving!"* and *"Status: Pass"*. Test with 95 (A), 60 (D, Pass), 59 (F, Fail), -5 and 105 (Invalid score). Reference: [labs/module-2/105-conditionals/solution/](#).

Q1

How many times does this loop run?

```
for (let i = 0; i < 3; i++) {  
  console.log(i);  
}
```

B) 3

Why: `i` takes 0, 1, 2; at 3 the condition `i < 3` is false and the loop exits.

Q2

Which loop should you use to iterate over an array's values?

B) `for...of`

Why: `for...of` hands you each value directly. `for...in` gives string keys and belongs to objects.

Q3

What does `continue` do inside a loop?

B) Skips to the next iteration

Why: the rest of the current pass is skipped and the loop carries on. `break` is the one that exits.

Q4

What is the key difference between `while` and `do...while` ?

D) `do...while` always runs its body at least once

Why: `do...while` checks its condition *after* each pass; `while` checks before, so it may run zero times.

Q5

What goes wrong with this loop?

```
const items = ["a", "b", "c"];
for (let i = 0; i <= items.length; i++) {
  console.log(items[i]);
}
```

B) It runs one extra time and logs undefined

Why: off-by-one. The last index is 2 but `<=` lets `i` reach 3, and `items[3]` is `undefined` (no error). Use `<` `items.length`.

EXERCISE SOLUTION — STAR RATING & SEARCH FILTER

Look for: `starRating` looping `i = 1; i <= 5` and appending `i <= rating ? "★" : "☆"`, then returning the string (1 → ★☆☆☆☆, 3 → ★★★☆☆, 5 → ★★★★★); a `for...of` over `products` with `product.toLowerCase().includes(searchTerm.toLowerCase())`; a `let matchCount` declared outside the loop; output of the three Wireless products and *"3 results found."* Reference: [labs/module-2/106-loops/solution/](#).

L07 · FUNCTIONS

Q1

What does a function return if it has no `return` statement?

B) undefined

Why: every call produces a value; with no `return`, that value is `undefined`. No error.

Q2

Which can be called before it's defined in the code?

A) Function declaration

Why: declarations are fully hoisted (name + body). `const` expressions and arrows are in the TDZ until their line runs.

Q3

What is the output of `const fn = n => n * 3; console.log(fn(4));` ?

B) 12

Why: a one-expression arrow returns its expression implicitly: 4×3 .

Q4

What does this code log?

```
function greet(name = "friend") {  
  return `Hi, ${name}!`;  
}  
console.log(greet());
```

C) "Hi, friend!"

Why: no argument means `name` would be `undefined`, so the default `"friend"` kicks in.

Q5

Which arrow function correctly returns an object?

D) `const make = n => ({ name: n });`

Why: without the parentheses, the braces are read as a function body, so the function returns `undefined`. Wrapping in `( )` makes them an object literal.

EXERCISE SOLUTION — MINI UTILITY LIBRARY

Look for: `capitalize` as a **declaration** using `str[0].toUpperCase() + str.slice(1).toLowerCase()` (with an empty-string guard); `clamp` as a **function expression** (`Math.max(min, Math.min(max, value))`); `pluralize` as an **arrow** with `plural = singular + "s"`; `temperatureConverter` returning `"32°F = 0.0°C"`, `"100°C = 212.0°F"`, and `"Invalid unit"` for anything else. Every helper must **return**, not **log**. Reference: [labs/module-2/l07-functions/solution/](#).

Q1

What is the output?

```
console.log(x);  
var x = 10;
```

B) undefined

Why: the `var` declaration is hoisted and initialized to `undefined`; the assignment of 10 stays on its own line.

Q2

Which keyword creates a block-scoped variable?

B) let

Why: `let` and `const` stay inside their `{ }`; `var` is function-scoped and leaks out of blocks.

Q3

What is a closure?

C) A function that remembers variables from its outer scope

Why: the inner function keeps access to its outer variables even after the outer function has finished — the basis of the private `count` in `createCounter`.

Q4

What happens when this code runs?

```
console.log(total);  
let total = 50;
```

B) ReferenceError: Cannot access 'total' before initialization

Why: `let` is hoisted but not initialized; the line is inside the Temporal Dead Zone.

Q5

What does this code log?

```
const color = "blue";
function paint() {
  const color = "red";
}
paint();
console.log(color);
```

D) "blue"

Why: the inner `color` is a separate, function-scoped variable that shadows the global only inside `paint()`. No redeclaration error: they live in different scopes.

EXERCISE SOLUTION — SCOPE DETECTIVE & SCORE TRACKER

Part 1 answers: 1 "block a" · 2 "global b" · 3 "function a" · 4 "function c" · 5 "function d" (`var` ignores the block) · 6 "global a" · 7 "global b" · 8 `ReferenceError` (`c` only exists inside `test()`). Part 2, look for: `let score = 0` inside `createScoreTracker`; a returned object with `addPoints`, `penalty` using `Math.max(0, score - points)`, `getScore` returning `score`, and `reset`. The usage sequence shows 30, then 0 after `reset()`; `tracker.score` is `undefined`. Reference: [labs/module-2/108-scope-closures/solution/](#).

Module 3 — Answers & Solutions

Correct quiz answers with the "why," plus a reference solution for each hands-on exercise. Use for grading and for reteaching missed items.

Lessons 9–12

17 quiz items

4 exercises

NUMBERING MATCHES THE WORKBOOK

This key covers every question in each lesson's quiz (four each, five in L12). The participant workbook's self-check uses a 3-question subset per lesson, labelled with the lesson's own question numbers (e.g. Q1 · Q3 · Q4), so a student's "Q4" is always this key's Q4.

L09 • ARRAYS

Q1

What is the index of the first element in a JavaScript array?

A) 0

Why: arrays are zero-indexed, so the first element is `arr[0]` and the last is `arr[arr.length - 1]`.

Q2

Which method adds an element to the END of an array?

B) push()

Why: `push` adds to the end and returns the new length. `unshift` adds to the start; `pop` removes from the end.

Q3

What does `splice()` do?

C) Removes, inserts, or replaces elements at any position

Why: `splice(start, deleteCount, ...items)` edits the original array in place. Copying part of an array (A) is `slice`.

Q4

What does `["a", "b", "c"].includes("b")` return?

B) true

Why: `includes` answers yes/no with a boolean. `indexOf("b")` would return the position, 1.

EXERCISE SOLUTION — TO-DO LIST MANAGER

Look for: `addTodo` uses `push` and returns the list; `removeTodo` uses `indexOf`, checks for `-1` (logging `Task not found: ...`), then `splice(index, 1)`; `insertTodoAt` uses `splice(index, 0, task)`; `listTodos` checks `length === 0` first and numbers from 1 with `forEach((task, i) => ...)`; `completeTodo` uses `shift`, logs `Completed: ...`, and handles the empty list. Final list after the tests: 1. Finish homework · 2. Call dentist · 3. Read a book. Reference: [labs/module-3/109-arrays/solution/](#).

L10 · OBJECTS

Q1

Which notation must you use when the key is stored in a variable?

B) Bracket notation

Why: bracket notation evaluates what's inside the brackets, so `obj[field]` uses the value stored in `field`. `obj.field` looks for a key literally named "field".

Q2

What does `Object.keys({ a: 1, b: 2, c: 3 })` return?

B) `["a", "b", "c"]`

Why: `Object.keys` returns the property names. Values (A) come from `Object.values`; pairs (C) from `Object.entries`.

Q3

Why shouldn't you use arrow functions as object methods that use `this`?

B) Arrow functions don't have their own `this`

Why: arrow functions inherit `this` from the surrounding scope, so inside an arrow "method" `this` isn't the object. Use the shorthand `greet() { }` syntax.

Q4

What does `{ x: 1 } === { x: 1 }` evaluate to?

B) false

Why: objects are compared by reference. These are two separate objects in memory, so they're not equal even with identical contents.

EXERCISE SOLUTION — CONTACT BOOK

Look for: `addContact` checks for a duplicate name with `find` before `push` and returns `true / false`; `findContact` returns `contact || null`; `updateContact` reuses `findContact` and applies `Object.assign(contact, updates)` (or spreads the updates in); `displayContact` prints `Name: Alice | Phone: 555-9999 | Email: alice@example.com | Group: friend` after the update; `listByGroup(contacts, "family")` → `["Bob", "Dave"]`. A `for...of` + `push` version of `listByGroup` is full credit, since `filter/map` come in L11. Reference: labs/module-3/l10-objects/solution/.

L11 · ARRAY METHODS

Q1

What does `map()` return?

B) A new array with the same number of elements

Why: `map` transforms each element and always returns a new array of the same length. Fewer items is `filter`; one value is `reduce`.

Q2

What does `[1,2,3].filter(n => n > 1)` return?

C) `[2,3]`

Why: `filter` keeps elements whose callback returns true: `2 > 1` and `3 > 1` pass; `1 > 1` is false.

Q3

What is the second argument to `reduce()` ?

B) The initial value of the accumulator

Why: `reduce(callback, initialValue)`. The initial value is the accumulator's starting point. Always pass one, or an empty array throws a `TypeError`.

Q4

Which method modifies the original array?

C) `sort()`

Why: `sort` reorders the array in place. `map`, `filter`, and `reduce` return new values and leave the original alone. Use `toSorted()` or `[...arr].sort()` to keep the original.

EXERCISE SOLUTION — PRODUCT CATALOG PIPELINE

Look for: no `for` / `while` loops. Expected results: (1) 8 names via `map`; (2) 6 in-stock products via `filter`; (3) [Mousepad XL, Wireless Mouse, Mechanical Keyboard] via `filter` → `sort (a, b) => a.price - b.price`; (4) "374.94" via `filter` → `reduce(..., 0)` → `toFixed(2)` (raw value 374.94000000000005); (5) `every(p => p.rating >= 4.0)` → `true`; (6) six lines, cheapest first, from "Mousepad XL — \$11.99 (was \$14.99)" to "Headphones — \$119.99 (was \$149.99)". Check that nobody sorted `products` itself before filtering. Reference: [labs/module-3/l11-array-methods/solution/](#).

L12 · STRINGS & TEMPLATE LITERALS

Q1

Are JavaScript strings mutable or immutable?

B) Immutable — methods always return new strings

Why: every string method returns a new string; the original never changes. `str[0] = "X"` does nothing (or throws in strict mode).

Q2

What does `"hello world".split(" ")` return?

B) `["hello", "world"]`

Why: `split` breaks the string at every space and returns an array of the pieces.

Q3

How do you check if a string ends with ".pdf"?

B) `str.endsWith(".pdf")`

Why: `endsWith` checks the end of the string, which is perfect for file extensions. `includes` would also match `"my.pdf.txt"`.

Q4

What does `"Hello, World!".slice(-6)` return?

D) "World!"

Why: a negative start counts from the end: the last 6 characters are `"World!"`.

Q5

What does `"7".padStart(3, "0")` return?

A) "007"

Why: `padStart` adds characters to the *start* until the string reaches length 3: `"007"`. The result is still a string.

EXERCISE SOLUTION — TEXT FORMATTING UTILITY

Look for: `truncate` returns the string untouched when it fits, else `slice(0, maxLength - 3) + "..."` (`"Hello..."` is 8 characters); `slugify` chains `trim` → `toLowerCase` → `replace` (strip specials, spaces → hyphens, collapse hyphens) for `"my-blog-post-1"`; `countWords` trims, returns 0 for an empty string, and splits on `/\s+/` → 3 for `" lots of spaces "`; `initials` splits, maps each word to its first letter, joins, uppercases → `"RDLP"`; `maskEmail` splits on `"@"` and uses `repeat` → `"r**@example.com"`. Regex use is expected for `slugify/countWords`; don't mark students down for asking about it. Reference: [labs/module-3/l12-strings/solution/](#).

Module 4 — Answers & Solutions

Correct quiz answers with the "why," plus a reference solution for each hands-on exercise. Use for grading and for reteaching missed items.

Lessons 13–16

21 quiz items

4 exercises

NUMBERING MATCHES THE WORKBOOK

This key covers every question in each lesson's quiz. The participant workbook's self-check uses a 3-question subset per lesson, labelled with the lesson's own question numbers (e.g. Q1 · Q4 · Q5), so a student's "Q4" is always this key's Q4.

L13 · INTRODUCTION TO THE DOM

Q1

What is the DOM?

B) An in-memory tree of objects representing the page

Why: the browser builds the DOM from the HTML; JavaScript works on that tree, never on the file.

Q2

Which object is the entry point to the DOM?

B) document

Why: `document` represents the whole page and holds `querySelector`, `createElement`, etc. `window` is the browser window around it.

Q3

What does `element.parentElement` return?

B) The element that contains this element

Why: one step up the tree — an `<li>`'s parent is its `<ul>` or `<ol>`.

Q4

Your JavaScript adds a new `<p>` to the page. Where will you see it?

C) In the DevTools Elements panel

Why: the Elements panel shows the live DOM. View Source and the file in VS Code show the original HTML, which JavaScript never changes.

Q5

For a `<ul>` written on several indented lines, why might `list.firstChild` not be the first `<li>` ?

D) The whitespace before the first `<li>` is a text node, and `firstChild` includes text nodes

Why: the line break and indentation become a text node. `firstElementChild` skips it.

EXERCISE SOLUTION — DOM EXPLORER

Look for: each console task answered on the worksheet — the title, the `h1`, the section count, the breadcrumb, a sibling walk from `document.body.firstElementChild`, the changed heading and background, the link count; `$0` used after selecting in the Elements panel; the bonus walk `documentElement.children[1].children[3]...` reaching the `<h1>`. For the "think about it" questions: `firstChild` is a whitespace text node (`nodeType` 3); View Source doesn't show console changes; editing in the Elements panel never changes the file. Reference: [labs/module-4/l13-dom-explorer/solution/](#) (runs every task and prints the results on the page).

L14 · SELECTING & MODIFYING ELEMENTS

Q1

What does `querySelector` return if no element matches?

C) null

Why: no error at selection time — the error comes on the next line, when you read a property of `null`. (`querySelectorAll` is the one that returns an empty NodeList.)

Q2

Why should you avoid `innerHTML` with user input?

B) It can execute malicious scripts (XSS attacks)

Why: `innerHTML` parses HTML, so `<img src=x onerror=...>` runs the attacker's code. `textContent` treats it as plain text.

Q3

What method adds a CSS class without removing existing ones?

A) `element.classList.add("new")`

Why: `className =` and `setAttribute("class", ...)` replace every class; `toggle` would remove it if it were already there.

Q4

Using the `dataset` property, how do you read `data-item-id` ?

B) `element.dataset.itemId`

Why: `dataset` converts kebab-case to camelCase. `dataset.item-id` is a subtraction; `getAttribute("data-item-id")` works but doesn't use `dataset`.

Q5

Your stylesheet sets `h1 { font-size: 2rem; }`. What does `heading.style.fontSize` return if no inline style was set?

D) `""` (an empty string)

Why: `element.style` reads only inline styles. `getComputedStyle(heading).fontSize` would give `"32px"`.

EXERCISE SOLUTION — DYNAMIC PROFILE CARD EDITOR

Look for: name, bio and location changed with `textContent` ; the link's text *and* `href` updated;
`classList.add("highlight")` ; `style.backgroundColor` set; `dataset.userId` logged (`"101"`);
`dataset.status = "active"` ; `classList.toggle("dark")` , with the inline background cleared
(`card.style.backgroundColor = ""`) so the `.dark` class can show — "prefer classes over inline styles."
Bonus: `updateProfile({ name, bio, location, website })` using destructuring and `if` checks per field. Reference: [labs/module-4/l14-profile-card/solution/](#).

L15 · EVENTS & EVENT LISTENERS

Q1

What does `addEventListener` require as its two arguments?

B) An event type string and a callback function

Why: `el.addEventListener("click", handler)`. An optional third argument holds options like `{ once: true }`.

Q2

What is `event.target` ?

B) The element that was actually clicked or interacted with

Why: the element the listener is attached to is `event.currentTarget` ; with bubbling the two can differ.

Q3

What does `event.preventDefault()` do?

C) Cancels the browser's built-in behavior for that event

Why: e.g. a form reloading the page or a link navigating. Stopping the bubbling is `stopPropagation()` , a different method.

Q4

What is the main advantage of event delegation?

C) One listener handles events for all current and future children

Why: events bubble up to the parent, so elements added later are handled with no extra code.

Q5

How can you make a listener fire only once?

B) `addEventListener("click", handler, { once: true })`

Why: the options object is the *third* argument; the browser removes the listener after it fires.

Q6

A delegated listener on a list checks `event.target.matches(".delete-btn")` , but clicking the  `<span>` inside the button does nothing. What's the fix?

A) Use `event.target.closest(".delete-btn")`

Why: the target is the `<span>` , which doesn't match. `closest()` checks the element itself and then walks up to the nearest matching ancestor.

EXERCISE SOLUTION — INTERACTIVE TASK LIST

Look for: a `submit` listener with `event.preventDefault()` ; trimmed input with an early `return` when empty; each `<li class="task-item">` built with fixed markup, the user's text set with `textContent` (never `innerHTML`); the input cleared and refocused; **one** click listener on `#task-list` using `event.target.closest()` to toggle `done` or `remove()` the task; `updateCounter()` called after every change. Bonus: Escape clears the input; "d" toggles `body.dark-mode` only when the input isn't focused. Reference: [labs/module-4/l15-task-list/solution/](#) .

L16 · CREATING & REMOVING ELEMENTS

Q1

What does `document.createElement("div")` return?

B) A new `<div>` element in memory, not yet in the DOM

Why: it's invisible until you insert it with `append` , `prepend` , `before` or `after` .

Q2

What happens if you `append` an element that's already in the DOM?

B) It moves from its old position to the new one

Why: a DOM element can only be in one place at a time. Use `cloneNode(true)` for a copy.

Q3

What is the modern way to remove an element from the DOM?

C) `element.remove()`

Why: no parent reference needed. `parentNode.removeChild()` is the older pattern; `display: none` only hides it.

Q4

What does `cloneNode(true)` do compared to `cloneNode(false)` ?

A) Deep clone (element + all children) vs. shallow clone (element only)

Why: `true` copies all descendants. Neither copies listeners added with `addEventListener` .

Q5

Why use a `DocumentFragment` when adding many elements?

C) It batches many insertions into one DOM update

Why: you build off-screen, then one `append` inserts all the children; the fragment itself disappears.

EXERCISE SOLUTION — DYNAMIC NOTIFICATION SYSTEM

Look for: `showNotification(type, message)` building a `div.notification.{type}` with `createElement`, the message in a `span.notification-text` via `textContent`, and a `button.close-btn` with "×" and `aria-label="Dismiss notification"`; `prepend` into `#notification-area` (which keeps `aria-live="polite"`); a 5-second `setTimeout` that removes it and updates the count; delegated listeners on `.controls` (`dataset.type`, or `replaceChildren()` for Clear All) and on the notification area (`closest(".close-btn")` → `closest(".notification").remove()`); `updateCount()` after every change. Bonus: the "b" key builds five notifications in a `DocumentFragment` and prepends it once. Reference: [labs/module-4/l16-notifications/solution/](#).

Module 5 — Answers & Solutions

Correct quiz answers with the "why," plus what to look for in each hands-on exercise and where to find the reference solution. Use it for grading and for reteaching missed items.

Lessons 17–20

24 quiz items

4 exercises

NUMBERING MATCHES THE WORKBOOK

This key covers every question in each lesson's quiz. The participant workbook's self-check uses 3 questions per lesson, labeled with the lesson's own question numbers (e.g. Q2 · Q4 · Q5), so a student's "Q4" is always this key's Q4.

L17 • FORMS & VALIDATION

Q1

How do you read what the user typed into a text input?

A) `input.value`

Why: `.value` is the current content of an input, textarea, or select, and it's always a string. `textContent` / `innerHTML` read the text inside ordinary elements, not what was typed.

Q2

What does `event.preventDefault()` do in a submit handler?

C) Stops the browser from reloading the page or navigating away

Why: by default a submit navigates to the `action` URL or reloads the page. `preventDefault()` keeps you on the page so JS can handle the data. Stopping bubbling is `stopPropagation()`.

Q3

How do you check if a checkbox is checked?

D) `checkbox.checked`

Why: `.checked` is a live boolean. `.value` is the value attribute (often `"on"`), and `getAttribute("checked")` only shows the starting state.

Q4

When is the `blur` event useful for form validation?

B) When you want to validate after the user finishes with a field and moves on

Why: `blur` fires when the user leaves a field. They've finished typing, so it's a fair moment to judge. Every keystroke is `input`, and entering a field is `focus`.

Q5

Why is client-side validation not sufficient on its own?

C) Users can bypass it through DevTools, disabling JS, or direct API calls

Why: anything running in the browser is under the user's control. Client-side checks improve the experience, but only server-side checks protect the data.

Q6

You call `email.setCustomValidity("Use your company email")`. The user fixes the address, but the form still won't submit. Why?

D) The custom message was never cleared with `setCustomValidity("")`

Why: any non-empty custom message keeps `validity.customError` true. Your code must reset it with an empty string once the value is acceptable.

EXERCISE SOLUTION — REGISTRATION FORM WITH REAL-TIME VALIDATION

Look for: `novalidate` on the form; `showError` reusing an existing `.error-message` (with `role="alert"`, set via `textContent`) rather than stacking new ones; four validators that each return `true` / `false`; a strength meter scoring 0–5 (length ≥ 8 , upper, lower, digit, special) that sets the bar width to `score/5 × 100%` and the label; a `touched` Set, so `blur` validates and `input` re-validates only touched fields; changing the password re-checks a touched confirm field; submit calls `preventDefault()`, runs *all* validators, and either hides the form and shows `#success-message` or focuses the first `.input-error`. **Test:** an empty submit focuses Username; a weak password reads "Very weak"; a mismatched confirm error clears after the password is changed to match. Ask students where the server re-validates (in the real `fetch("/api/register")` endpoint). Reference: [labs/module-5/l17-forms-validation/solution/](#).

Q1

In the browser, what does `setTimeout` return?

D) A numeric timer ID you can pass to `clearTimeout()`

Why: the callback hasn't run yet when `setTimeout` returns. What you get back is the timer's ID, which is the handle you use to cancel it.

Q2

What will `console.log("A"); setTimeout(() => console.log("B"), 0); console.log("C");` output?

B) A, C, B

Why: even with a 0 ms delay, the callback waits in the task queue until the running code (the call stack) finishes, so "C" logs before "B."

Q3

What's the difference between `setInterval` and recursive `setTimeout` ?

C) `setInterval` fires every N ms regardless of how long the work takes; recursive `setTimeout` waits for completion first

Why: slow async work inside an interval can overlap. A recursive timeout schedules the next run only after the current one finishes, which guarantees a gap.

Q4

Why use `requestAnimationFrame` instead of `setInterval` for visual animations?

A) It syncs with the display's refresh rate for smooth frames and pauses in background tabs

Why: rAF runs right before each repaint (60/120/144 fps) and stops when the tab is hidden, which saves CPU and battery. `setInterval` has no idea when the screen redraws.

Q5

What is debouncing?

D) Delaying a function until events stop firing, resetting the timer on each new event

Why: each new event resets the timer, so the function runs once after a pause. Option A describes throttling.

Q6

In what order do `setTimeout(..."timer"..., 0)`, `Promise.resolve().then(..."promise"..., console.log("sync"))` log?

B) sync, promise, timer

Why: synchronous code runs first, then the browser empties the microtask queue (promise callbacks) before it takes the next task (the timer). The order they were scheduled in doesn't decide the order they run in.

EXERCISE SOLUTION — POMODORO TIMER WITH ANIMATED PROGRESS

Look for: state variables declared *outside* the functions; `updateDisplay` formatting MM:SS with `padStart(2, "0")` and setting `strokeDashoffset = CIRCUMFERENCE × (1 - remaining/total)`, where `CIRCUMFERENCE = 2π × 130 ≈ 816.81`; `start()` guarded by `if (isRunning) return` so a double-click doesn't create two intervals; the interval cleared and the done message shown at 0; pause/reset clearing the interval; presets for 25/15/5/1 min that pause, reset, and move the `selected` class; bonus ring colors of red under 30 s and green at 0. **Stretch:** an `endTime` set with `Date.now()` and remaining time recalculated each tick; `prefers-reduced-motion` turns off the ring transition. Reference: `labs/module-5/l18-timers-animation/solution/` (lesson version) and `.../stretch/`; event-loop demo in `.../demo/event-loop.html`.

L19 · LOCAL STORAGE

Q1

What does `localStorage.getItem("key")` return if the key doesn't exist?

B) `null`

Why: a missing key gives `null`, not `undefined` or `""`, and it never throws. That's why loaders use `stored ? JSON.parse(stored) : fallback`.

Q2

What happens when you store a JavaScript object directly with `setItem`?

D) It's stored as the string `"[object Object]"`

Why: storage only holds strings, so the object is converted with `.toString()` and the data is lost. Use `JSON.stringify` first.

Q3

What is the correct way to store an array in Local Storage?

A) `localStorage.setItem("data", JSON.stringify(myArray))`

Why: JSON keeps the structure and types (nested objects, numbers, booleans). `JSON.parse` turns it back into an array. Passing the array directly gives a flat `"1,2,3"`.

Q4

How does `sessionStorage` differ from `localStorage`?

C) `sessionStorage` data is cleared when the tab is closed; `localStorage` persists until deleted

Why: same API and a similar ~5 MB limit. The difference is lifetime and scope: session data belongs to one tab, while local data is shared across tabs of the same origin.

Q5

Why should you never store passwords or auth tokens in Local Storage?

C) Any JavaScript on the page can read it, making it vulnerable to XSS attacks

Why: an injected script can read every key. Auth tokens belong in HttpOnly cookies, which JavaScript can't read.

Q6

Your to-do app is open in two tabs. In Tab A you add a task (`setItem("todos", ...)`). Where does the `storage` event fire?

B) In Tab B (and any other open tabs of the same origin), but not in Tab A

Why: the tab that made the change already knows about it. The event goes to the *other* tabs, which is what makes cross-tab sync possible.

EXERCISE SOLUTION — PERSISTENT TO-DO LIST

Look for all 7 TODOs: 1 `loadTodos` with try/catch returning `[]` for missing or corrupted data, and `saveTodos` using `JSON.stringify`; 2 `render` clears the list, shows an empty state, and builds each `li.todo-item` (checkbox, `span.todo-text` via `textContent`, `button.delete-btn` "x", a `done` class, `data-id`); 3 `addTodo` trims and ignores blanks, then pushes `{ id: Date.now(), text, done: false }`; 4 toggle with `map` and delete with `filter`, each followed by save + render; 5 a pluralized "X items left" count; 6 click + Enter to add, **one delegated listener** on `#todo-list` using `closest(".todo-item")` and `Number(li.dataset.id)`, and "Clear completed"; 7 an initial `render()`. **Test:** add, tick, refresh, and the data is still there. **Credit for polish:** `aria-label`s on the checkbox and x button (the lesson's version has none) and a label for the input. The reference includes them. Reference: [labs/module-5/119-local-storage/solution/](#).

L20 · WORKING WITH APIS & FETCH

Q1

What does `fetch()` return?

B) A Promise that resolves to a Response object

Why: the data isn't there yet. The Response comes first, and `response.json()` is a second Promise that gives you the data.

Q2

What happens when `fetch()` gets a 404 response?

A) The Promise resolves normally with `response.ok` set to `false`

Why: fetch only rejects for network failures. HTTP errors are still valid responses, so check `response.ok` or `response.status` yourself.

Q3

What does the `async` keyword do to a function?

C) Makes the function always return a Promise and enables the use of `await` inside it

Why: `async` doesn't create a thread or speed anything up. It wraps the return value in a Promise and allows `await` inside the function.

Q4

Why use `Promise.all()` instead of multiple sequential `await` calls?

B) It runs requests in parallel, so the total time equals the slowest request rather than the sum of all

Why: sequential awaits start each request only after the previous one finishes. `Promise.all` starts them all at once.

Q5

What are the three UI states every fetch operation should handle?

D) Loading, data (success), and error

Why: these are what the user sees. Pending/resolved/rejected are the Promise's internal states.

Q6

Your page on `http://127.0.0.1:5500` fetches from another site's API, and the console shows a CORS error. What's going on?

C) The API server didn't send a header allowing your origin, so the browser won't let your code read the response

Why: permission comes from the server's `Access-Control-Allow-Origin` header. `fetch` rejects with a `TypeError`, and the fix belongs on the server (or a proxy of your own), not in your `fetch` options.

EXERCISE SOLUTION — GITHUB USER SEARCH

Look for: DOM references; `showLoading` (spinner + "Searching..."); `showError` that escapes the message; `showUser` built with `createElement` + `textContent` (avatar with alt text, name or "No name", `@login`, bio or "No bio available", repos/followers/following); `async searchUser` that trims and ignores blanks, uses `encodeURIComponent`, and branches `404` → "not found", `403/429` → rate-limit message, other `!ok` → the status, and `catch` → "Network error"; click + Enter listeners; bonus profile link (`html_url`, `rel="noopener"`) and debounced auto-search. Classroom additions in the reference: a `USE MOCK` switch (or `?mock`) that serves `mock-user.json` when the room is rate-limited, `AUTO_SEARCH` off by default, and a `latestSearch` guard against stale responses. Reference: `labs/module-5/l20-apis-fetch/solution/`.

Module 6 — Answers & Solutions

Correct quiz answers with the "why," plus reference notes for the QuickNote capstone and the Snack Budget bug hunt. All quizzes are from the course lessons.

Lessons 21-22

10 quiz items

2 labs

NUMBERING MATCHES THE WORKBOOK

This key covers every question in each lesson's quiz. The participant workbook's self-check uses a 3-question subset per lesson, labelled with the lesson's own question numbers (e.g. Q1 · Q2 · Q4), so a student's "Q4" is always this key's Q4.

L21 · BUILDING A COMPLETE PROJECT

Q1

QuickNote uses one form for both creating and editing notes. How does the submit handler know which one to do?

B) It checks whether the hidden `note-id` input has a value

Why: The hidden `note-id` input is empty for a new note and holds the note's ID when editing, so one if/else picks `createNote()` or `updateNote()`.

Q2

Why does `renderNotes()` put the note's title and body on the page with `textContent` instead of `innerHTML` ?

C) So anything the user typed is shown as plain text and never run as HTML or script

Why: `textContent` always treats the text as plain text, so anything a user types — even something that looks like an HTML tag or script — is displayed, never run.

Q3

What does wrapping the search handler in `debounce(applyFilters, 300)` accomplish?

A) The notes re-render only after the user stops typing for 300ms, not on every keystroke

Why: Each keystroke restarts a 300ms timer, so the notes are only filtered and re-rendered once the user pauses typing.

Q4

Why does QuickNote attach ONE click listener to `#notes-grid` instead of a listener on every edit and delete button?

D) The cards are rebuilt on every render, so one listener on the parent (event delegation) handles every card, old or new

Why: This is event delegation. `renderNotes()` throws away and rebuilds every card, so listeners on individual buttons would be lost. One listener on the parent catches clicks from all current and future cards.

Q5

What happens in QuickNote if the quote API is down or the user is offline?

B) A fallback quote is shown and the rest of the app keeps working

Why: The try/catch in `fetchQuote()` catches the failure, shows a built-in fallback quote, and everything else keeps working. That's graceful degradation.

EXERCISE SOLUTION — BUILD QUICKNOTE

Reference: `labs/module-6/l21-quicknote/solution/quicknote.html`, the lesson's complete source code, unchanged. Catch-up files after Steps 3–6: `labs/module-6/l21-quicknote/checkpoints/`. Look for:

- **Data layer with no DOM code:** `loadNotes` in try/catch returning `[]`; `saveNotes` after every change; `createNote` uses `Date.now()` ids, trims text, `unshift` s; `getNotes` matches category *and* lowercase search.
- **Rendering:** clear → empty state → cards built in a `DocumentFragment`, appended once; title and body set with `textContent`; count reads "1 note" / "2 notes".
- **Form:** hidden `#note-id` decides create vs update (`Number()` on the id); both error messages appear on an empty save, each right after its field; errors cleared before re-validating; status message restarts its 2-second timer with `clearTimeout`.
- **Search & filter:** `input` → `debounce(applyFilters, 300)`; `change` → `applyFilters`.
- **Quote:** `res.ok` checked; the `catch` shows the fallback quote, so the app still works offline.
- **Polish:** one delegated listener on `#notes-grid`; deleting the note open in the form calls `hideForm()`; N and / ignored while typing or with Ctrl/Cmd/Alt; ESC always closes.

Extensions (the 8 stretch challenges) are described with hints in `labs/module-6/l21-quicknote/README.md`; there is no single reference solution. Grade them against the same test checklist plus the new feature.

L22 · NEXT STEPS & BEST PRACTICES

Q1

The console shows `Cannot read properties of null (reading 'textContent')`. What's the most likely cause?

C) `querySelector` didn't find the element, so it returned `null`

Why: `querySelector` returns `null` when nothing matches — usually a typo in the selector, or the script ran before the element existed. Reading `.textContent` on `null` throws this error.

Q2

A player's score is `0`. What does `const shown = player.score ?? 10; set shown` to?

A) `0`

Why: `??` only falls back when the left side is `null` or `undefined`. `0` is a real value, so it's kept. (With `||` you'd get `10`, because `0` is falsy.)

Q3

You copy an object with `const copy = { ...original };` and then push a new item onto `copy.scores` (an array). What happens to `original.scores` ?

B) It changes too, because the nested array is shared

Why: Spread makes a SHALLOW copy: both objects share the same scores array. Use `structuredClone(original)` when you need a deep copy.

Q4

You want to start a new React project today. Which approach fits current best practice?

C) Use Vite (`npm create vite@latest`) or a framework like Next.js

Why: Vite (`npm create vite@latest`) or a framework like Next.js is the standard starting point now that Create React App is deprecated.

Q5

You add a `debugger;` line to your code and reload, but nothing pauses. What's the most likely reason?

B) DevTools wasn't open, so the browser ignored it

Why: The browser ignores debugger statements unless DevTools is open. Open DevTools (F12), then reload the page.

EXERCISE SOLUTION — BUG HUNT: SNACK BUDGET

The lesson has no build exercise; this lab practices its error table and pitfalls. Fixed version: `labs/module-6/l22-debugging/solution/` (each fix marked `// FIX n` in `script.js`). The bugs, in the order the Console reveals them:

#	SYMPTOM	CAUSE → FIX
1	<code>SyntaxError: Unexpected token '{' — nothing runs</code>	Missing comma after the first object in <code>seedItems</code> , on the line <i>before</i> the one reported → add the comma.
2	<code>Cannot read properties of undefined (reading 'weekly')</code>	First visit: no saved settings, so <code>settings.limits</code> is undefined → <code>settings.limits?.weekly ?? 20</code> .
3	<code>Cannot read properties of null (reading 'replaceChildren')</code>	Selector typo <code>"#item-lsit"</code> → <code>"#item-list"</code> .
4	<code>ReferenceError: formatprice is not defined</code>	Case-sensitive name → <code>formatPrice</code> .
5	<code>TypeError: items.length is not a function</code>	<code>length</code> is a property → <code>items.length</code> .
6	No error: Bought works on the first click, then never again	The delegated listener is added inside <code>render()</code> , so each render stacks another copy and an even number of toggles cancels out → move <code>listEl.addEventListener(...)</code> out of <code>render()</code> so it runs once.
7	Clicking Load sample items: <code>Cannot read properties of undefined (reading 'forEach')</code>	Async trap: <code>samples</code> is used before <code>fetch</code> finishes → make the handler <code>async</code> , <code>await fetch(...)</code> and <code>await response.json()</code> , check <code>response.ok</code> , and catch errors.

Look for: a clean Console; Bought/Undo toggling every time with Spent/Left updating; three sample items added; deletes and reloads working. Accept any correct fix. For bug 2, `settings.limits && settings.limits.weekly` plus a default is fine; for bug 7, `.then()` with the dependent code moved inside it is fine. A `setTimeout` "fix" for bug 7 is *not*: it only hides the race. Follow-on projects (the workbook menu) are self-directed and have no key.